Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет “Высшая школа экономики”

Факультет компьютерных наук
Дисциплина «Архитектура компьютера и операционные системы»

ОТЧЁТ
по индивидуальному заданию № 1

Вариант 31
«Управление взлётно-посадочными полосами»

Выполнил: Шепурев Степан Сергеевич
Группа: БПИ252
Дата: 07.10.2026

Москва, 2026

Содержание

1. Постановка задачи и критерии выполнения

2. Принятые модельные решения

3. Упрощённая структура программы

4. Алгоритм последовательной имитации

5. Диспетчеризация, топливо и закрытие ВПП

6. Пользовательский интерфейс и ввод-вывод

7. Инварианты и обеспечение корректности

8. Тестирование и результаты

9. Критическая оценка решения и использования ИИ

10. Инструкция по сборке и запуску

11. Заключение

1. Постановка задачи и критерии выполнения

Вариант 31 требует разработать консольное приложение, моделирующее работу диспетчерской службы аэропорта с одной или несколькими взлётно-посадочными полосами. Самолёты поступают с запросами на посадку или взлёт, имеют уникальный номер, тип, требуемую длину полосы, длительность операции и категорию приоритета. Прибывающий самолёт в ожидании расходует топливо; вылетающий ожидает на земле. Диспетчер назначает совместимую свободную ВПП с учётом интервала безопасности. Критически малый остаток топлива повышает приоритет, но уже начатая операция не прерывается.

В исходном условии отдельно перечислены входные параметры, обязательные отображаемые события, инварианты и способы завершения. Кроме того, указано, что диспетчер, каждая ВПП и каждый самолёт являются самостоятельными участниками модели. В реализации самолёты и полосы представлены структурами Plane и Runway, а диспетчер — отдельной структурой Dispatcher. Поведение участников реализовано обычными функциями последовательной симуляции.

Группа требований	**Что должно присутствовать в варианте 31**	**Реализация**
Входные параметры	ВПП; число ARRIVAL/DEPARTURE; типы; интервалы заявок; длительности; топливо; safety; приоритеты; закрытия	Конфигурационный файл + CLI-переопределения; FLOW и AIRCRAFT
События	Запрос, очередь, изменение топлива, назначение, начало/окончание, освобождение, emergency, alternate, закрытие	Все события имеют отдельные теги журнала
Инварианты	Одна операция на ВПП; совместимость; safety; операция не повторяется; явный исход при топливе	Автоматическая проверка после каждого такта
Завершение	Все заявки; конец смены; прерывание в unlimited-режиме	Три штатных причины завершения + отдельный аварийный код для нарушения инварианта

Документ с требованиями к ИЗ устанавливает C и Linux как обязательную основу, информативный консольный вывод и отчёт. К дополнительным возможностям, повышающим оценку, отнесены комментарии, ввод-вывод через файловые дескрипторы, корректное прерывание, лог, тестовые наборы и скрипты, публикация ИИ-промптов и критическая оценка сгенерированного кода. Все эти пункты реализованы.

Дополнительный пункт	**Что сделано**
Дескрипторный ввод-вывод	Журнал открывается через open(), вывод в stdout и файл выполняется через write(), закрытие — close().
Остановка и прерывание	SIGINT/SIGTERM: простой обработчик signal() выставляет sig_atomic_t-флаг, завершение выполняется в основном цикле.
Лог	Полная трасса событий и итоговая статистика в отдельный файл.
Тестовые данные и скрипты	6 CTest-сценариев, отдельная SIGINT-проверка и scripts/run_tests.sh.
ИИ-промпты	report/AI_PROMPTS.md.
Критическая оценка ИИ	Раздел 9 данного отчёта.
Дополнительные решения	Инвариант-чекер, проверка уникальности номеров, отдельный Dispatcher, best-fit ВПП, воспроизводимый seed, статистика, planned closures и санитайзер-проверка.

2. Принятые модельные решения

Моделирование дискретное, шаг времени — один такт. Все длительности операций, интервалы поступления, расход топлива, safety interval и интервалы закрытия задаются в тактах. Это позволяет однозначно проверять причинно-следственный порядок событий и воспроизводить эксперимент с тем же seed.

- ВПП совместима с самолётом, если length_m >= required_length_m.
- Операция, начавшаяся в такт T и имеющая duration=D, заканчивается в такт T+D; в этот момент ВПП освобождается до работы диспетчера.
- После завершения в T новая операция разрешена не раньше T+safety_interval. При safety=0 полоса может быть назначена в тот же такт завершения.
- Прибывающий самолёт расходует топливо только если после работы диспетчера он всё ещё находится в WAITING. Это исключает искусственный расход при немедленном назначении.
- Критический уровень задаётся как burn_rate × EMERGENCY_FUEL_TICKS. При достижении или пересечении порога самолёт получает аварийный приоритет.
- Если ожидающий ARRIVAL после расхода топлива достигает 0, регистрируется DIVERT и уход на запасной аэродром. Топливо никогда не становится отрицательным.
- Плановое закрытие имеет полуинтервал [start,end). Если в start на полосе уже идёт операция, она продолжается, но новые назначения блокируются; после окончания полоса закрывается до end.



Рисунок 1 — автомат состояний самолёта

3. Упрощённая структура программы

Файл	Ответственность
src/airport.h	Структуры Plane, Runway, Dispatcher и общие константы/объявления.
src/main.c	Параметры модели, CLI, обработка сигналов и запуск симуляции.
src/config.c	Чтение конфигурации, генерация FLOW и проверка уникальности номеров самолётов.
src/io.c	Информативный вывод в терминал и журнал через файловые дескрипторы.
src/simulation.c	Такт модели, диспетчеризация, топливо, закрытия, инварианты, события завершения и статистика.
configs/	Основной сценарий и тестовые конфигурации, включая duplicate и interrupt.
scripts/run_demo.sh	Сборка и наглядный демонстрационный запуск.
scripts/run_tests.sh	Сборка, CTest, сценарные grep-проверки, duplicate test и SIGINT test.
report/	Отчёт, результаты тестов и неизменённый файл AI_PROMPTS.md.

CMakeLists.txt	Сборка четырёх .c-файлов и шесть CTest-сценариев.
README.md	Краткая инструкция по структуре, сборке, запуску и конфигурации.

Под «самостоятельным участником» здесь понимается отдельная структура данных со своим состоянием. ОС-процессы и потоки не создаются: первое ИЗ посвящено последовательной имитации. Код разбит на несколько небольших учебных модулей: airport.h содержит структуры и общие объявления, main.c — запуск и CLI, config.c — чтение конфигурации и создание самолётов, io.c — вывод и журнал, simulation.c — основную логику модели.

4. Алгоритм последовательной имитации

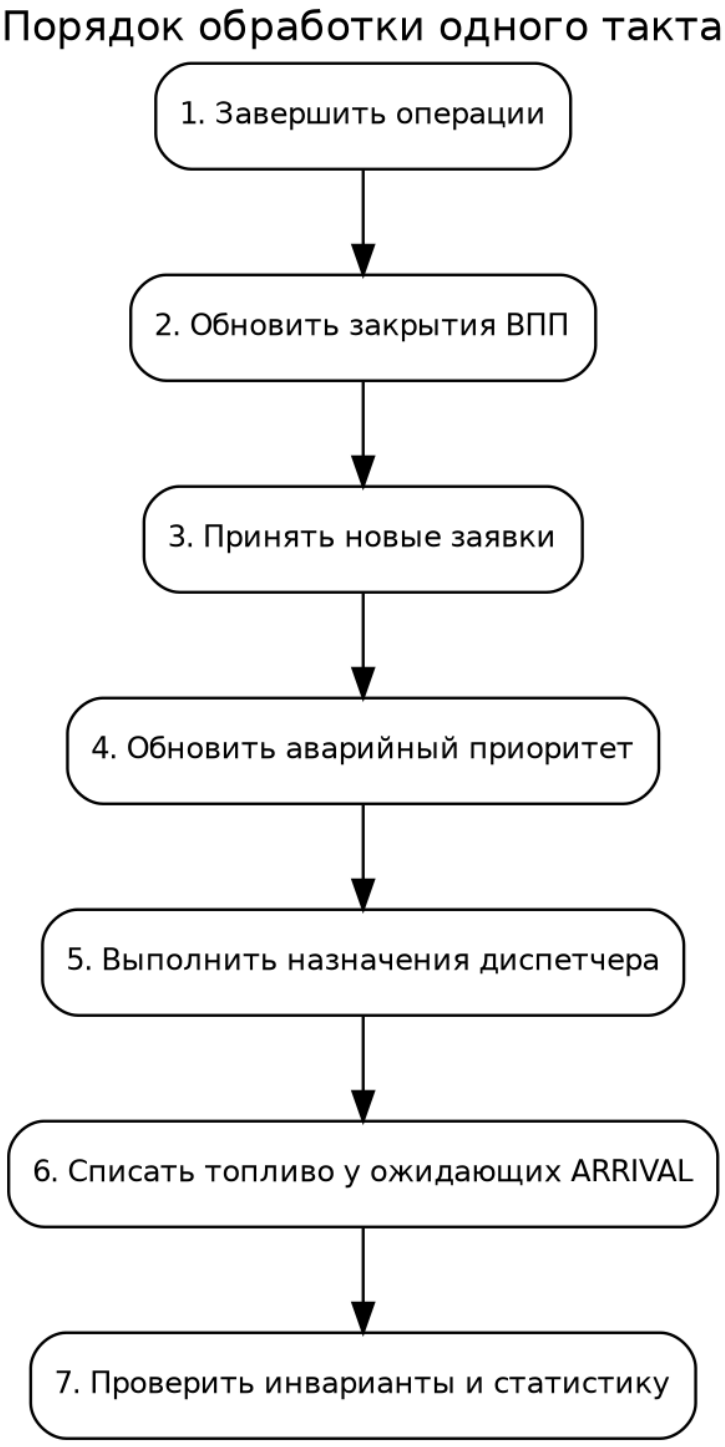


Рисунок 3 — порядок обработки событий внутри такта

Один проход главного цикла соответствует одному такту. Сначала завершаются операции, затем печатаются события закрытия/открытия ВПП, принимаются новые заявки, определяется аварийный приоритет, выполняется диспетчеризация и только после этого расходуется топливо у оставшихся в очереди самолётов. Такой порядок легко проследить прямо в функции run_simulation().

1. finish_operations(): завершает посадки/взлёты, освобождает ВПП и задаёт next_free_tick с учётом safety interval.
2. print_closure_events(): выводит CLOSE/OPEN; фактический запрет нового назначения проверяется функцией runway_closed(). Уже начатая операция не прерывается.
3. register_requests(): переводит PENDING -> WAITING и печатает REQUEST + QUEUE.
4. update_emergencies(): повышает приоритет ожидающих ARRIVAL при критическом топливе.
5. dispatch_planes(): выбирает самолёт с наивысшим приоритетом и подходящую свободную ВПП.
6. burn_fuel_and_wait(): уменьшает топливо только у ARRIVAL, которые после диспетчеризации остались WAITING; при нуле выполняет DIVERT.
7. check_invariants(): проверяет основные инварианты; затем проверяются условия завершения модели.

5. Диспетчеризация, топливо и закрытие ВПП

5.1. Очередность самолётов

Приоритет сравнивается по простому порядку: (1) аварийное состояние по топливу; (2) категория HIGH > NORMAL > LOW; (3) более ранний request_tick; (4) более раннее положение самолёта в массиве. Аварийный ARRIVAL обгоняет обычный HIGH, но не прерывает уже идущую операцию.

5.2. Выбор полосы

Для выбранного самолёта рассматриваются только свободные, не закрытые и не находящиеся в safety interval ВПП достаточной длины. Из них выбирается полоса с минимальным запасом длины. Стратегия best fit снижает вероятность занять длинную ВПП лёгким самолётом, когда короткая также подходит.

5.3. Плановые закрытия

Закрытия задаются директивой CLOSE и проверяются по текущему такту. В интервале [start,end) функция runway_closed() запрещает новые назначения на эту ВПП. Если в момент start операция уже идёт, она просто продолжается до обычного завершения. В end выводится OPEN.

6. Пользовательский интерфейс и ввод-вывод

Основной интерфейс — CLI и текстовый журнал. Конфигурационный файл намеренно выбран человекочитаемым: каждый параметр задаётся одной строкой, комментарии начинаются с #. Это позволяет формировать тестовые наборы без перекомпиляции.

```
SEED 31
MAX_TICKS 120
EMERGENCY_FUEL_TICKS 3
RUNWAY RWY-A 3400 2
TYPE A320 1800 3 3
CLOSE RWY-A 28 38
FLOW ARRIVAL 16 0 1 4
FLOW DEPARTURE 14 0 1 5
```

Конфигурация читается обычными учебными функциями fopen/fgets/sscanf. Для выполнения дополнительного требования по файловым дескрипторам журнал открывается системным вызовом open(), а вывод в stdout и лог выполняется через write(); файл закрывается close(). При добавлении

самолёта отдельно проверяется уникальность его номера: повторный AIRCRAFT с тем же номером считается ошибкой конфигурации и моделирование не запускается.

Пример запуска:

```
./build/airport_sim --config configs/default.conf --seed 31 --delay-ms 120 --log
logs/demo.log
```

Формат события содержит такт, тип события и предметное описание:

```
[T=0002] [EMERGENCY] A001 has critical fuel=3; emergency priority enabled
[T=0004] [ASSIGN   ] ATC assigned A001 to R1 (EMERGENCY)
[T=0004] [START    ] A001 started LANDING on R1; finish at T=6
```

Теги покрывают события из условия: REQUEST, QUEUE, FUEL, ASSIGN, START, FINISH, RELEASE, EMERGENCY, DIVERT, CLOSE. Дополнительно реально выводятся OPEN и три события завершения: COMPLETE при обработке всех заявок, SHIFT-END при окончании смены и STOP при SIGINT/SIGTERM. После события завершения печатается итоговая статистика.

6.1. Варианты завершения

- ALL_REQUESTS_COMPLETED — все самолёты получили конечный исход.
- SHIFT_ENDED — достигнут MAX_TICKS; в отчёте выводятся WAITING/OPERATING/PENDING.
- USER_INTERRUPT — в режиме без ограничения времени (MAX_TICKS=0) пользователь нажал Ctrl+C; также обрабатывается SIGTERM.
- INVARIANT_FAILURE — защитное завершение, если внутренняя проверка обнаружила противоречие; код возврата 3.

7. Инварианты и обеспечение корректности

Вместо того чтобы только предполагать соблюдение инвариантов, программа проверяет их после каждого такта. Дополнительно уникальность номеров проверяется уже при добавлении самолёта, а затем повторно контролируется инвариант-чекером. Это даёт простой механизм самоконтроля модели и удобный аргумент при защите работы.

Инвариант	Проверка в программе
Одна ВПП — не более одного самолёта	В Runway хранится только один plane_index; при busy проверяется обратная ссылка самолёта.
Самолёт только на совместимой ВПП	best_runway_for_plane() проверяет длину, затем check_invariants() повторно проверяет совместимость.
Safety interval соблюдён	После FINISH задаётся next_free_tick = current_tick + safety; до этого такта ВПП не назначается.
Операция выполняется не более одного раза	times_started увеличивается при START; check_invariants() запрещает значение больше 1.
Критическое топливо имеет явный исход	EMERGENCY при достижении порога; DIVERT при fuel == 0.
Согласованность состояний	RUNNING самолёт и BUSY ВПП обязаны ссылаться друг на друга.
Топливо неотрицательно	После вычитания значение ограничивается нулём и дополнительно проверяется инвариантом.
Уникальность номера самолёта	config.c отклоняет повторный номер при add_plane(); check_invariants() повторно сравнивает номера всех

	самолётов.

Если проверка не проходит, событие INV-FAIL записывается до завершения. В штатных тестах такие события отсутствуют.

8. Тестирование и результаты

Модульная версия проверена обычной сборкой, CTest, shell-скриптом с grep-проверками и отдельной сборкой с AddressSanitizer + UndefinedBehaviorSanitizer. Все проверки прошли успешно.

Сценарий	Что проверяет	Наблюдаемый результат
test_nominal.conf	Совместимость, параллельная работа 2 ВПП, best fit, обычное завершение	2 посадки + 2 взлёта, 0 DIVERT, ALL_REQUESTS_COMPLETED.
test_emergency.conf	Аварийное повышение приоритета без прерывания текущей операции	A001 становится EMERGENCY и в T=4 назначается раньше ожидающего HIGH D002.
test_closure.conf	Закрытие во время BUSY	В T=2 печатается CLOSE с сообщением, что текущая операция продолжается; полоса снова доступна после OPEN в T=8.
test_incompatible.conf	Несовместимая длина, истощение топлива, конец смены	ARRIVAL уходит на запасной; DEPARTURE остаётся WAITING и выводится в summary.
test_safety.conf	Точный safety interval	После finish T=2 и safety=2 следующий START происходит T=4; затем T=8.
SIGINT test	Unlimited mode и корректное прерывание	Печатается STOP, затем termination=USER_INTERRUPT; код возврата 130, журнал закрыт штатно.
test_duplicate.conf	Уникальность номера самолёта	Повторный A001 отклоняется до запуска модели с сообщением Duplicate aircraft number: A001.

Результат CTest: 6/6 tests passed. Результат scripts/run_tests.sh: All tests passed. Скрипт дополнительно проверяет COMPLETE, SHIFT-END, STOP, корректный код 130 после SIGINT и отклонение дублирующегося номера самолёта. Отдельная сборка с ASan/UBSan также прошла пять основных тестовых конфигураций без сообщений об ошибках.

8.1. Основной демонстрационный сценарий (seed = 31)

```
termination=ALL_REQUESTS_COMPLETED, elapsed_ticks=64, last_tick=63
aircraft_total=30, landed=10, took_off=14, diverted=6, waiting=0, operating=0,
pending=0
emergency_escalations=7, completed_avg_wait=5.33 ticks
dispatcher=ATC assignments=24 emergency_assignments=1
runway=RWY-A operations=9 busy_ticks=41 utilization=64.1%
runway=RWY-B operations=10 busy_ticks=28 utilization=43.8%
runway=RWY-C operations=5 busy_ticks=14 utilization=21.9%
```

Шесть уходов на запасной в основном сценарии не являются ошибкой программы: это допустимый исход при случайном потоке, ограниченной пропускной способности, закрытиях и конечном запасе топлива. Каждый такой исход явно фиксируется событием DIVERT.

9. Критическая оценка решения и использования ИИ

При разработке использовалась помощь ИИ для анализа требований, исправления ошибок, подготовки README, разработки тестовых сценариев, анализа вывода и оформления отчётных материалов. Перечень рабочих промтов вынесен в report/AI_PROMPTS.md.

9.1. Правки и проверки

- Уточнён порядок «диспетчер -> расход топлива»: самолёт, получивший ВПП в текущем такте, не должен сначала терять топливо как ожидающий.
- Проверена граница safety interval: next_free_tick = finish + safety. Для finish=T2, safety=2 тест требует START=T4, а не T5.
- Добавлена защита от отрицательного топлива и явный DIVERT при достижении нуля.
- Сохранён компактный check_invariants после каждого такта: он проверяет совместимость ВПП, согласованность ссылок, запрет повторного старта самолёта и уникальность номеров. Кроме того, дубликат номера отклоняется сразу в config.c при добавлении самолёта.
- SIGINT приводит к событию STOP, итоговому termination=USER_INTERRUPT и коду возврата 130.

10. Инструкция по сборке и запуску

Требуется Linux, CMake и компилятор C с поддержкой C11.

```
cmake -S . -B build
cmake --build build
./build/airport_sim --config configs/default.conf --log logs/run.log
```

Наглядный запуск с задержкой:

```
./scripts/run_demo.sh
```

Автоматическая проверка:

```
./scripts/run_tests.sh
```

Санитайзеры:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -Isrc \
    src/main.c src/config.c src/io.c src/simulation.c -o airport_sim_asan
./airport_sim_asan --config configs/test_nominal.conf --no-log --quiet
```

11. Заключение

Разработана последовательная имитационная программа для варианта 31. Реализованы все предметные события и инварианты из задания, три штатных способа завершения, дескрипторный вывод, журнал, статистика, воспроизводимые случайные потоки, конфигурация, тестовые сценарии и обработка сигналов. Диспетчер выделен в отдельную структуру Dispatcher, номера самолётов проверяются на уникальность, а события COMPLETE, SHIFT-END и STOP действительно фиксируются в журнале.

Проект ориентирован на уровень 8 по формальным критериям, но без излишнего усложнения. Код разделён на понятные учебные модули main.c, config.c, io.c и simulation.c с общим заголовком airport.h. Дополнительно сохранены конфигурационный файл, журнал, обработка Ctrl+C, проверка инвариантов, best-fit выбор ВПП, плановые закрытия, статистика и набор сценарных тестов.

Приложение А. Соответствие обязательных событий тегам журнала

Событие из условия	Тег
Запрос операции	REQUEST
Постановка в очередь	QUEUE
Изменение топлива	FUEL
Назначение ВПП	ASSIGN
Начало операции	START
Окончание операции	FINISH
Освобождение ВПП	RELEASE
Аварийный приоритет	EMERGENCY
Уход на запасной аэродром	DIVERT
Закрытие полосы	CLOSE

Приложение Б. Структура проекта

```
airport_runway_sim/
├── CMakeLists.txt
├── README.md
├── src/
│   ├── airport.h
│   ├── main.c
│   ├── config.c
│   ├── io.c
│   └── simulation.c
├── configs/
│   ├── default.conf
│   └── test_*.conf
├── scripts/
│   ├── run_demo.sh
│   └── run_tests.sh
└── report/
    ├── AI_PROMPTS.md
    ├── test_results.txt
    └── Отчёт - Шепурев Степан Сергеевич.docx / .pdf
```